



Lecture 2: Setting up a Svelte Project

INTERVIEW QUESTION

[> CORE](#)

How do you create and run a new Svelte project?

You want to start a Svelte app today. What command do you run, what does it give you, and how do you see it in the browser?

Starting a new project in modern Svelte is done entirely through a dedicated command-line tool. Unlike the early days where you might have manually wired up a bundler, the Svelte team now provides an official, zero-configuration pathway.

The Modern Standard: `sv create`

The easiest and official way to create a Svelte project is to use the `sv` CLI. This tool generates a complete, ready-to-run skeleton for your application.



When starting a new project, you need a place to put all your files, your settings, and your codebase. Should we manually create every single folder? Should we create an `index.html`, then a `styles.css`, and manually link them together one by one? This is a decision about how we build the foundation of our app. We need a way to instantly generate the correct folders and baseline files so we don't make mistakes setting up the architecture. This automated generation of the foundation is called **scaffolding** an application. When you run `npx sv create`, you are scaffolding a **SvelteKit** application.

KEY TAKEAWAY

SvelteKit is not an optional add-on; it is the official operating system for Svelte.

SvelteKit is the official framework built *on top of* Svelte (powered by Vite under the hood). To understand why it exists, we must look at what a full app needs.

When you create a full app, you need to make some decisions on how to present new pages. Should we name new pages automatically with "p", for example, `/p/1` is page 1, and `/p/2` is page 2? Or maybe we should write `/page-1` for page 1? This is a decision about the architecture of our app. We have to make sure that when a specific path is visited by the user in the browser (e.g., `mywebsite.com/page/1`), a specific part of our app is activated. This matching mechanism is called the **router** of the app. The router makes sure to match all the paths with the proper components.

SvelteKit provides this router for you. Just as an operating system provides the file system and networking for software, SvelteKit provides the architecture for your Svelte components to live in.

Furthermore, when a user clicks a link to go to a new page, the browser usually talks to a server, downloads a brand new HTML document, and the screen flashes white while loading. Do we always want that flash? What if we just download the entire website's structure once, and then use JavaScript to swap out the content instantly when the user clicks a link? This is a decision about how our app behaves. We need a way for the app to feel as fast as a desktop program without constantly asking the server for new pages. An app that loads once and swaps content dynamically without refreshing is called a **Single-Page Application (SPA)**.

Even if you only want to build a simple SPA and don't know the rest of SvelteKit yet, the team recommends using the `npx sv create` command. You can safely ignore the advanced features until you need them.

INTERVIEW TIP

To impress the interviewer: Mention that `npx sv create` scaffolds a SvelteKit project by default. Point out that the ecosystem has shifted away from standalone Svelte templates, and that SvelteKit is now the recommended baseline even for simple client-side apps.

The Development Server

After running `npm run dev`, Vite spins up a local web server (usually at `http://localhost:5173`). This server watches your files for changes. When you edit a component, Vite leverages Hot Module Replacement (HMR) to instantly push the updated compiled code to your browser without requiring a full page refresh.

The Standalone Alternative

If you absolutely must avoid SvelteKit (perhaps you are integrating Svelte into a legacy backend that serves its own HTML), you can use Vite directly:



A terminal window with a title bar containing three window control buttons and the label "Terminal". The terminal shows a single command: `1 npm create vite@latest myapp -- --template svelte`. Below the command, a callout box contains the text *Creates a*, a **STANDALONE** badge, and *Svelte setup*.

```
1 npm create vite@latest myapp -- --template svelte
```

Creates a **STANDALONE** *Svelte setup*

This bypasses SvelteKit entirely, giving you a bare-bones Svelte project. However, you will have to handle routing and server-side rendering yourself.

Summary

To start a new Svelte project, run `npx sv create myapp` . This command scaffolds a complete application structure powered by SvelteKit and Vite. After installing dependencies with `npm install` , you use `npm run dev` to start a local development server that instantly reflects your code changes in the browser.

	SVELTE CLI (<code>sv create</code>) (SvelteKit Powered)	VITE CLI (<code>create vite</code>) (Standalone SPA)
Output	Full-stack SvelteKit app skeleton	Client-side only Svelte skeleton
Routing	Built-in file- based routing	None (requires external library)
Recommendation	Official team recommendation for all apps	Useful for legacy integrations or strict SPAs